LEVROMART · ENGINEERING HANDOVER

Frontend Build Manual

How the Levromart app (the part users see) is built, how every screen talks to the Supabase backend, and how to change, test, ship and eventually migrate it. Written to be read next to the Backend Build Manual.

Prepared 7 October 2026 · Repository `kenniskalu18-prog/medris` · Companion: *Backend Build Manual* (PDF)
Source of truth for this manual: `index.html`, `sw.js`, `api/*.js`, `vercel.json`

Contents

PART 1 **Start here: orientation**

About this manual 4

1. What the frontend is 5

Technology 5

Roles the UI must handle 5

2. Repository map & deployment 6

Deployment 6

Vercel rewrites in vercel.json 6

3. Running, editing, testing and shipping 7

Run it locally 7

Find your way around 7

Check your edit before pushing 7

Test visually 7

Suggested workflow 7

PART 2 **How the frontend is built**

4. Anatomy of index.html 8

The page, top to bottom 8

Rule 1: later wins (CSS) 8

Rule 2: later wins (JavaScript) 8

Rule 3: ids and data-* are an API 8

Rule 4: nothing is hard-coded data 8

5. Start-up and core runtime 9

What happens when the app opens 9

Global state 9

The safe-fetch layer (read cache and friendly errors) 9

Other runtime pieces 9

Browser storage keys 10

6. Routing and navigation 11

All views 11

Deep links 12

7. UI toolkit 13

PART 3 How the frontend uses the backend

8. How the frontend talks to the backend 14
The contract in four rules 14
9. Authentication and accounts 15
Sign-in 15
Sign-up 15
Admin login 15
Passwords 15
After sign-in 15
10. Screen-by-screen backend map 16
Home, search and browsing 16
Product, package and storefront 16
Cart and buying 16
Orders (customer and vendor) 17
Messages and calls 17
Vendor tools 17
Admin 18
Leaderboard, referrals and Levi 19
Everywhere 19
11. Orders and payments in the UI 20
Status to button 20
The tracker 20
24-hour completion 20
Cart checkout 20
12. Realtime, push and presence 21
Realtime channels 21
Push notifications 21
Voice calls 21
13. Files and images 22
14. Levi, the AI assistant 23

PART 4 Lookup tables

15. Tables the browser touches directly 24
16. Every backend function the frontend calls 25

PART 5 The handover: recipes, migration, known issues

17. Quick recipes 27
18. Frontend migration playbook (leaving Supabase) 28
What binds the frontend to Supabase 28
Recommended path 28
If you choose a full rewrite of the data layer 28
Cut-over test checklist 28
19. Known issues and technical debt 29
Gotchas learnt the hard way 29
Appendix A: JavaScript section index 30

About this manual

1 HTML file holds the whole app	**53k** lines in index.html	**~1,500** JavaScript functions	**80** script blocks	**207** style blocks	**0** build steps
60+ screens (views)	**474** table queries	**224** RPC calls	**147** distinct RPCs used	**11** realtime channels	**9** /api endpoints used

How to read this manual

It is in five parts, in the order you need them. **Part 1** gets you running. **Part 2** explains how the app is built, in the order the browser runs it: page layout, start-up, navigation, UI helpers. **Part 3** is the heart: how the frontend uses the backend (sign-in, every screen's data, every action, realtime, files, Levi). **Part 4** is the lookup material. **Part 5** is the handover: moving off Supabase, known problems, recipes.

Whenever this manual says *the backend*, the detail lives in the Backend Build Manual. Section numbers there are written like "Backend §6.1".

The three ideas that explain everything:

1. **One file, no build.** The browser downloads `index.html` and runs it. There is no compiler, bundler or framework. What you edit is what runs.
2. **Later code wins.** The file grew by adding blocks at the end. A later CSS rule or a later redefinition of a function replaces an earlier one. To change behaviour you almost always edit the *last* definition.
3. **The browser holds no secrets.** It talks to Supabase with a public key; Row Level Security and database functions decide what it may do. Anything secret runs in `api/*.js`.

1. What the frontend is

Levromart is a mobile-first marketplace for Nigeria. Anyone can buy; verified vendors can also sell products, services and packages (bundles). It is a **Progressive Web App**: it opens in a browser, can be installed on a phone's home screen, works partly offline and can receive push notifications.

Technology

Layer	What is used	Notes
Language	Plain JavaScript (ES2020), HTML, CSS	No TypeScript, no framework, no bundler.
Backend client	`@supabase/supabase-js@2` loaded from a CDN	Global object `sb`. Used for tables, RPC, auth, storage and realtime.
Icons / fonts	Font Awesome 6.5.1, Inter (Google Fonts)	Both from CDNs.
Maps	Leaflet 1.9.4 + OpenStreetMap tiles	Loaded on demand (`MAP` section).
PDF	jsPDF 2.5.1 (CDN, loaded on demand)	Admin table exports only.
Offline / push	`sw.js` service worker	Caches the app shell; shows push notifications.
Hosting	Vercel (static file + 12 serverless functions)	Deploys on every push to `main`.
Images	Cloudflare R2 (public) and Supabase Storage (private)	See section 13.

Roles the UI must handle

Role	How the UI knows	What changes
Visitor	No `session`	Can browse; protected screens ask for sign-in.
Buyer	`profile.role = "buyer"`	Cart, orders, chat, follow, wishlist, leaderboard.
Vendor	`profile.role = "vendor"` and a row in `vendors` (global `vendor`)	Everything a buyer has, plus My Shop, listings, vendor orders, earnings. Locked out of tools when commission is overdue.
Admin	`profile.role = "admin"` (some actions need `is_super_admin`)	Admin panel: vendors, users, disputes, finance, conversations, points, content.
Suspended	`profile.suspended_at` is set	Every route is replaced by the Suspended screen with an appeal form.

> **Important:** the UI hides things by role for convenience only. The real rule is always enforced in the database (Backend §5). Never rely on a hidden button as security.

2. Repository map & deployment

Path	What it is	Touch it when
`index.html`	The entire frontend (HTML, CSS, JS).	Any UI or behaviour change.
`sw.js`	Service worker: shell cache, runtime cache, push display, notification clicks, call actions.	Push or caching changes. **Bump `CACHE` version** (currently `levromart-shell-v11`) so phones pick up the change.
`manifest.json`, icons	PWA name, colours, icons (192/512, maskable).	Branding changes.
`notif-icons/`, `levromart-*.png`	Notification icons and badge.	New notification type icon.
`api/*.js`	12 Vercel serverless functions (Backend §10).	Anything needing a secret.
`lib/compress-image.js`	Client-side image compression (the `.ts` files are reference copies, not used).	Image size/quality rules.
`vercel.json`	Rewrites, cron, function settings.	New route or scheduled call.
`.github/workflows/`	Weekly encrypted database backup; optional 30-minute order reminder trigger.	Backup/reminder changes.
`supabase/`, `docs/`	Database notes and exports; older text manual; native-calls plan.	Reference only.

Deployment

1. Commit and push to `main`. Vercel builds automatically; there is nothing to compile.
2. If the change must reach installed apps quickly, change the version string in `sw.js` (`CACHE` and, if runtime caching changed, `RUNTIME`).
3. Phones fetch the new service worker next time the app is opened and update after a reload. Browsers may need two opens.
4. Database changes are *not* deployed by Vercel. Apply SQL in Supabase first (to both projects if the standby is kept), then deploy the code that needs it.

Vercel rewrites in `vercel.json`

Public path	Served by	Why
`/api/turn`	`api/delete-account.js?turn=1`	WebRTC TURN/ICE servers for voice calls.
`/api/register`, `/api/account-exists`	`api/delete-account.js`	Sign-up and “does this email/phone exist” (kept in one file to stay under 12 functions).
`/api/listing-payments`	`api/vendor-payments.js`	Older name for the vendor-payments router.
`/?storefront=...`, `/?product=...`	`api/delete-account.js`	Returns the same `index.html` with link-preview meta tags filled in, so WhatsApp/Facebook previews show the real vendor or product.

Cron: `/api/daily-cron` at 03:00 UTC. Variables the *frontend* needs: none. The only values in `index.html` are public by design: `SUPABASE_URL`, `SUPABASE_ANON_KEY` (publishable key) and `VAPID_PUBLIC_KEY`. The full secrets list is in Backend §2.

3. Running, editing, testing and shipping

Run it locally

```
cd Medris
python3 -m http.server 8765        # any static server works
# open http://localhost:8765/index.html
```

The app then talks to the **live** Supabase project (the keys are in the file). Use a test account, and be careful with anything that writes. API calls such as `/api/notify` or payments need Vercel, so they will not work on a plain static server; use a Vercel preview deployment for those.

Find your way around

Every JavaScript area starts with a banner: `// ---------- NAME ----------`. A **CODE MAP** comment at the top of the file lists them all. Use search for the exact banner text. The full banner list with line numbers is in Appendix A. Functions are global, so search for a function name to find its definition(s); **the last definition wins** (section 4).

Check your edit before pushing

There is no compiler to catch typos, and one syntax error in any script block can break the whole app. Always syntax-check every inline script after editing:

```
python3 - <<'EOF'
import re,subprocess,tempfile,os
s=open('index.html').read()
for i,m in enumerate(re.finditer(r'<script(?![^>]*\bsrc=)[^>]*>(.*?)</script>',s,re.S)):
    f=tempfile.mktemp(suffix='.js'); open(f,'w').write(m.group(1))
    r=subprocess.run(['node','--check',f],capture_output=True,text=True)
    if r.returncode: print('block',i,r.stderr[:300])
EOF
for f in api/*.js; do node --check $f; done
```

Test visually

The repository has no automated tests. Changes were checked with Playwright (headless Chromium) at phone size (430×900), in light and dark mode, by loading the page from a local server, removing the splash and onboarding overlays, then taking screenshots. Keep doing that for any visual work; it is quick and catches the most common problem in this codebase, a rule being overridden by a later rule (section 4).

Suggested workflow

1. Pull the latest `main` first. Other people also edit `index.html`, so conflicts are real.
2. Make the smallest change that works. Add new CSS and JS at the **end** of the relevant block (section 4) with new prefixed names.
3. Syntax-check, then look at the screen in light and dark mode.
4. Push to `main`. Reload an installed app to see it (bump `sw.js` if needed).

> **Do not** move the first `<style>` block (it sits above `<!DOCTYPE html>`) without testing everything: moving it changes how the browser renders the page.

4. Anatomy of `index.html`

The page, top to bottom

Region	Contents
Before `<!DOCTYPE>`	One early `<style>` block (kept where it is on purpose).
`<head>`	Meta tags, manifest link, CDN scripts (Supabase, Font Awesome), the first large CSS and a few small scripts such as the read cache (`__levroSafeFetch`).
`<body>` shell	Empty containers: `#app` (every screen is rendered into it), bottom navigation, toast area, drawers, call overlay, Levi button, splash screen, onboarding overlay.
Script blocks (80)	The application. The first big one (from about line 10,250) contains the banners in Appendix A. Later blocks are patches and features added over time.
Style blocks (207)	Base styles first, then a long chain of “polish” layers. Named blocks have an `id` such as `native-app-shell-css` .

Rule 1: later wins (CSS)

The stylesheet is a stack of layers. When two rules match, the later one wins unless an earlier one has more specificity or `!important` . Many older rules use `!important` , and dark mode is applied with `:root[data-theme="dark"] …` selectors that are very specific. So a new rule often seems to do nothing. The fix is higher specificity at the end of the file, for example:

```
:root:root:root:root:root:root body #app .my-class { color: var(--text) !important; }
```

Check the result in both themes. Always give new things **new prefixed class names** so they cannot collide with old styles:

Prefix	Area	Prefix	Area
`vl-`	My Listings	`vd-`	Vendors directory
`od-`	Order details (and the tracker)	`sr-`	Search rows
`cart-`	Cart	`vh-`	Seller hub (My Shop)
`msg-`	Messages	`earn-`	Earnings
`va-`	Vendor application	`lv-`	Newer shared pieces (profile panels, sheets)
`cc-`	Confirm / bottom-sheet drawers	`admin-`	Admin screens

Rule 2: later wins (JavaScript)

All functions are global. Several were redefined later in the file, either as another `function` with the same name or as `window.name = …` . The browser keeps only the last one. Known multiples (so you edit the live one):

`submitVendorApply (5)` `renderVendorApply (4)` `navigate (wrapped 4+ times)` `setupMessageSwipeReply (4)` `vendorCardHtml (3)` `fetchLeviReply (3)` `subscribeToConversation (3)` `handleChatFileSelected (3)` `adminDeleteVendor (3)` `setVendorStatus (3)` `renderProductDetail (2)` `renderVendorsDirectory (2)` `renderStorefront` `renderVendorDashboard` `sendChatMessage (2)` `reportDirectVendorPayment (2)`

> **Trap:** editing the first definition does nothing. Search for the name, find the *last* occurrence, edit that one. This cost real time more than once (the vendor application page was edited in an old copy). A good clean-up task is removing the dead copies (section 19).

Rule 3: ids and `data-*` are an API

Much of the JavaScript finds elements by `id` or `data-*` attributes. Search before renaming either.

Rule 4: nothing is hard-coded data

Counts, names and prices come from Supabase. Do not paste sample values into the page.

5. Start-up and core runtime

What happens when the app opens

1. The browser shows the splash and loads the CDN scripts. `sw.js` is registered (`navigator.serviceWorker.register("/sw.js")`).
2. `const sb = supabase.createClient(URL, ANON_KEY, {global:{fetch: __levroSafeFetch}})` creates the one backend client (`SUPABASE` section).
3. **INIT**: the app restores the cached session (`sb.auth.getSession`), loads the user's row from `users` into the global `profile`, and the vendor row into `vendor`. A listener, `sb.auth.onAuthStateChange`, repeats this on sign-in, sign-out and token refresh.
4. The last route is restored (`PERSISTENT ROUTE + SCROLL STATE`) or the URL decides (`?storefront=`, `?product=`, `?ref=` referral code), then `navigate(view, params)` renders into `#app`.
5. Background helpers start: live notifications, live orders, presence heartbeat, push registration, legal-consent check, review prompt, first-visit tour.

Global state

Variable	Meaning
`sb`	The Supabase client. Every backend call goes through it.
`session`	Current auth session (access token, user id). Null when signed out.
`profile`	Row of `public.users` for the signed-in user (role, name, suspended_at, …).
`vendor`	Row of `vendors` owned by the signed-in vendor.
`currentView`, `currentViewParams`	Which screen is showing and with what parameters.

The safe-fetch layer (read cache and friendly errors)

Supabase calls pass through `window.__levroSafeFetch` (an early script in `<head>`). It does two jobs. First, it **caches identical GET reads** of slow-changing tables in memory for a short time, merges identical in-flight requests, clears the cache after any write, and serves the last good copy if the backend is down or over quota. This is the main protection against Supabase bandwidth costs. Second, when the backend answers with a quota, paused or gateway error it replaces the raw message with one friendly sentence, so every `error.message` shown to users is readable.

Table	Cache time	Table	Cache time
sectors, categories, points_rules	10 min	homepage_carousel_slides	5 min
products, vendors, vendor_packages, package_items, product_variants	25 s	product_rating_stats	40 s
product_/package_engagement_settings	60 s	Stale copy used when backend fails	up to 1 h

> The cache only touches URLs that match `https://*.supabase.co/rest/v1/<table>`. If the backend moves, this pattern must change (section 18).

Other runtime pieces

Piece	What it does
Offline catalogue cache	Stores the last useful home/browse data in `localStorage` (`levromart-offline-home-v2`, valid 7 days) so the app still browses offline.
Persistent route	Saves current view and scroll in `localStorage` (`levromart_persisted_route_v4`) so a reload returns to the same place.
Presence heartbeat	Every ~20 s calls `touch_presence` while the app is visible and `clear_presence` when it closes; `touch_last_seen` about every 45 s feeds "last seen". The server uses it to suppress pushes while the app is open (Backend §6.6).
Theme and accessibility	Theme (`medris-theme`, applied as `data-theme` on `<html>`), accent colour, font scale (`levromart-font-scale`), Levi button size and visibility.

Install prompt	“Install as app” banner; dismissal stored as `medris-install-dismissed`.
First-visit tour	Walks the real pages once; `levromart-tour-seen`.

Browser storage keys

Preference and cache keys only; nothing sensitive is stored. `medris-theme` `levromart-font-scale` `medris-levi-visible` `medris-levi-fab-size` `levi-voice` `medris-location-enabled` `medris-distance-auto` `levro-ref-code` `levromart_vendors_view` `levromart-vendor-listing-tab` `levromart-tour-seen` `medris-install-dismissed` `levromart_review_completed_v1` `medris-ai-last-conversation` `levromart-offline-home-v2` `levromart_persisted_route_v4` `lv_session_only`

6. Routing and navigation

There is no router library. `navigate(view, params, pushHistory)` (`NAVIGATION` section) is the single entry point. It has been wrapped several times by later features, but the logic is the same:

1. **Gate.** `levroCommissionGate(view)` : for vendors with an overdue commission it blocks My Shop, Edit Profile, Listings, Bundles, Suggestions and Settings and shows a payment screen (Backend §6.4). A suspended profile is forced to the `suspended` view.
2. **Protected views.** A fixed list (`protectedRouteViews`) needs a session. If auth is still restoring, navigation waits briefly instead of flashing “Access denied”.
3. **Render.** A `switch (view)` calls the matching `renderX()` , which returns an HTML string. That string replaces the inside of `#app` ; then page wiring (event handlers, loading data) runs.
4. **History and scroll.** The browser history gets an entry; Home scroll position and profile scroll are remembered so Back returns to the same place.

Screens are **string templates**: a render function builds HTML with template literals, always passing user text through `esc()` , and buttons call global functions through `onclick="..."` . To add or change a screen you edit a `renderX` function and its helpers.

All views

View name	Render function	Who	What it shows
`home`	`renderHome`	Public	Home: banner carousel, category rail, trending, featured vendors, browse-everything with infinite scroll.
`search`	`renderSearchPage`	Public	Search with All / Products / Vendors / Services / Packages tabs.
`productDetail`	`renderProductDetail`	Public	Product or service page: photos, variants, reviews, comments, buy / add to cart / name your price.
`packageDetail`	`renderPackageDetail`	Public	Bundle page.
`storefront`	`renderStorefront`	Public	Vendor’s public shop.
`vendors`	`renderVendorsDirectory`	Public	Vendors list and map.
`cart`	`renderCart`	Signed in	Cart grouped by vendor; checkout per vendor or all.
`wishlist`	`renderWishlist`	Signed in	Saved products and packages.
`messages`	`renderMessages`	Signed in	Inbox, chat, call log rows, chat lists.
`notifications`	`renderNotificationsPage`	Signed in	Notification list.
`buyerOrders`	`renderCustomerOrders`	Signed in	Customer’s orders.
`orderHistory`	`renderOrderHistoryPage`	Signed in	Order history with search and tabs.
`orderDetail`	`renderOrderDetail`	Signed in	Order page (customer or vendor view).
`leaveReview`	`renderLeaveReview`	Signed in	Review form for a completed order.
`buyerProfile`	`renderBuyerProfile`	Signed in	Profile hub.
`editProfile`	`renderEditProfile`	Signed in	Edit own details.
`settings`	`renderSettingsPage`	Signed in	Settings: theme, notifications, account, delete account.
`customerFollowing`	`renderCustomerFollowing`	Signed in	Vendors the user follows.
`requestEquipment`	`renderRequestEquipment`	Signed in	Post a request for an item.
`supportRequest`	`renderSupportRequest`	Signed in	Support ticket form.
`leaderboard`	`renderLeaderboard`	Signed in	Points, ranks, referral.
`vendorApply`	`renderVendorApply`	Signed in	Become a vendor.
`vendorDashboard`	`renderVendorDashboard`	Vendor	My Shop (seller hub, stats).

`vendorEditProfile`	`renderVendorEditProfile`	Vendor	Shop details, billing model.
`vendorProducts`	`renderVendorProducts`	Vendor	My Listings.
`vendorPackages`	`renderVendorPackages`	Vendor	My bundles.
`vendorOrders`	`renderVendorOrders`	Vendor	Customers' orders (sales).
`vendorEarnings`	`renderVendorEarnings`	Vendor	Sales totals and charts.
`vendorFollowers`	`renderVendorFollowersPage`	Vendor	Followers.
`vendorRequests`	`renderVendorRequests`	Vendor	Customer requests to answer.
`vendorSuggestionsInbox`	`renderVendorSuggestionsInbox`	Vendor	Suggestions sent to the vendor.
`adminPanel`	`renderAdminPanel`	Admin	Vendor approvals, locked vendors, trash.
`adminUsers`	`renderAdminUsers`	Admin	Users and roles.
`adminPeople`	`renderAdminPeople`	Admin	Appeals and suspended people.
`adminDisputes`	`renderAdminDisputes`	Admin	Disputes and reported payments.
`adminFinance`	`renderAdminFinance`	Admin	Transactions and payouts.
`adminConversations`	`renderAdminConversations`	Admin	Conversations (Active / Earlier).
`adminMonthlyListings`	`renderAdminMonthlyListings`	Admin	Monthly-listing receipts to approve.
`adminPoints`	`renderAdminPoints`	Admin	Points adjustments and social awards.
`adminSupportRequests`	`renderAdminSupportRequests`	Admin	Support tickets.
`adminFeedback`	`renderAdminFeedback`	Admin	Suggestions.
`adminPasswordResets`	`renderAdminPasswordResets`	Admin	Manual password-reset requests.
`adminLog / adminAuthLogs`	`renderAdminLog / renderAdminAuthLogs`	Admin	Action log and login log.
`adminCarousel / adminLinkLibrary`	`renderAdminCarousel / …`	Admin	Home banners and link library.
`adminR2Migration / adminR2Optimize`	`…`	Admin	One-time image migration tools.
`adminInventory`	`renderAdminInventory`	Admin	Legacy inventory screen.
`terms / privacy / refundPolicy / contactUs`	`renderTerms …`	Public	Legal and contact pages.
`sharedAiChat`	`renderSharedAiChat`	Public link	A shared Levi conversation.
`suspended`	`renderSuspendedScreen`	Suspended	Appeal screen; replaces every other screen.

Deep links

Notifications carry `target_view` and `target_params` (for example `orderDetail` with `{orderId}`). Tapping one calls `navigate(target_view, target_params)`, so the database decides where a notification leads. Share links use `/?storefront=<id>` and `/?product=<id>`; referral links use `?ref=<code>` (kept in `levro-ref-code` until sign-up).

7. UI toolkit

Helpers you will use constantly

Helper	Use
`byId(id)`	Short for `document.getElementById`.
`esc(text)`	Escapes text for HTML. **Always** wrap database text in it before putting it in a template (prevents script injection).
`naira(n)`, `effectivePrice(o)`, `priceHeroInner`	Money formatting and the price a buyer really pays (discount when valid).
`showToast(message, type)`	Small message at the top. Types: info, success, error.
`showConfirmDialog(message, opts)`	Replaces browser `confirm()`. Returns a promise.
`openDrawer(id)`, `closeAllPanels()`	Slide-in panels and bottom sheets. The sheets hide the bottom navigation while open.
Long-press selection	WhatsApp-style multi-select for chats and notifications.
Map helpers	Leaflet loaded on demand; vendor locations and “View on map” in order details.
Image compress	`lib/compress-image.js` reduces photos in the browser before upload.
Modal back-button handling	Back button and outside-click close open modals instead of leaving the page.

Patterns used by most screens

- **Instant shell, then data.** A screen often renders a skeleton at once, then fills it after the queries return. Snapshots (chats, orders, notifications) are cached so repeat opens feel instant.
- **Batching.** Instead of one call per row, the app batches (for example `get_orders_buyer_names`, `get_vendor_follower_counts`, `get_favorite_counts`). Keep doing this: round trips cost time and money.
- **Debounce + abort.** Search waits for typing to pause and cancels stale requests; results cache for 60 s.
- **Infinite scroll** uses `IntersectionObserver`, 24 items per page.
- **Dark mode** comes from CSS variables (`--text`, `--bg`, …) plus `:root[data-theme="dark"]` overrides. Test every new element in both.

8. How the frontend talks to the backend

Everything the browser does with the backend goes through one of six doors. Counting call sites in `index.html`:

Door	How it is called	Call sites	Backend reference	Used for
Tables (REST)	`sb.from("table").select/insert/update/delete/upsert`	474	Backend §3, §5	Reading lists and simple writes. Protected by RLS.
RPC	`sb.rpc("function", args)`	224	Backend §8	All business actions: orders, payments, follows, chat, admin. Run as SECURITY DEFINER and check `auth.uid()`.
Auth	`sb.auth.*`	31	Backend §2, §5	Sign in, sign out, session, password reset, update user.
Storage	`sb.storage.from(bucket)`	27	Backend §9	Private files (chat attachments, documents, receipts, evidence).
Realtime	`sb.channel(...).on("postgres_changes" | broadcast)`	11 channels, 17 listeners	Backend §6.6, §6.7	Live notifications, orders, messages, comments, call signalling.
Vercel API	`fetch("/api/...")`	40	Backend §10	Anything secret: payments, email/push, Levi AI, sign-up, image upload signing.

The contract in four rules

- **Reads** are plain queries; RLS decides which rows come back. A forbidden row is simply missing, not an error.
- **Writes that matter** go through an RPC. The frontend never sets order status, payment status, money amounts or roles directly; it calls a function that checks who is asking and enforces the rules (Backend §6).
- **Secrets never appear in the browser.** The browser sends its access token to `/api/*`; the function verifies it and uses the service key.
- **Errors:** Supabase returns `{data, error}`. The pattern is `const {data, error} = await sb.rpc(...); if (error) { showToast(error.message, "error"); return; }`. Many RPCs raise readable messages on purpose (for example “Chat is only open around an order”), so show `error.message` as it is.

> **Why this matters for a migration:** the frontend depends on the *shape* of Supabase (PostgREST query syntax, GoTrue auth, Realtime channels, Storage API), not just on a database. Section 18 explains the options.

9. Authentication and accounts

Sign-in

- Users sign in with **email or phone number plus password** (`submitAuth` → `sb.auth.signInWithPassword`).
- Supabase Auth only knows emails. A phone number is converted to a private alias: digits normalised (`08012345678` → `2348012345678`) and turned into `p<digits>@accounts.levromart.app` (`authAliasFromPhone`). That is the real login identity for phone users. Keep this when migrating.
- Google sign-in exists as an option (`signInWithOAuth`).

Sign-up

The browser calls `POST /api/register` (served by `api/delete-account.js`), which creates the auth user and the `users` row with the service key, then the app signs in. `/api/account-exists` checks whether an email or phone is already used. Referral codes: `?ref=` is stored as `levro-ref-code` ; `validate_referral_code` checks it at sign-up and `apply_referral_code` records it (Backend §6.8).

Admin login

Admins also get a one-time email code. After the password step the app calls `POST /api/auth-otp` (`checkOtpRequirement` , `submitLoginOtp` , `resendLoginOtp`); the codes live in `admin_login_otps` , written and checked only by the server.

Passwords

- Normal reset: `sb.auth.resetPasswordForEmail` .
- Manual reset for people without usable email: the user files a request, an admin approves it (`admin_approve_manual_password_reset`), then the user sets a new password (`complete_manual_password_reset`). Request state lives in `levromart_manual_password_reset` (browser) and `password_reset_requests` (database).
- `setAccountPassword` / change password go through `/api/auth-otp` .

After sign-in

Step	Call	Purpose
Load profile	`sb.from("users").select()`	Fills `profile` .
Sync role	`sync_current_account_role`	Makes sure `users.role` matches the vendor/admin rows.
Log activity	`record_auth_activity`	Feeds the admin login log.
Legal consent	`get_legal_terms_consent` / `accept_legal_terms`	Shows terms again when the version changes (bump the version string in these functions).
Push	`push_subscriptions` upsert	Registers the device for web push (VAPID public key in the file).
Suspension	`profile.suspended_at`	Forces the Suspended screen; `submit_account_appeal` / `get_my_account_appeal` handle appeals.
Delete account	`POST /api/delete-account`	Removes the user and their data (server side).

10. Screen-by-screen backend map

For each area: which **tables the screen reads**, and which **RPC / API calls its actions make**. This was produced by tracing the code, then checked against the function list. Shared calls that run on almost every screen are listed once at the end ("Everywhere"), not repeated.

Home, search and browsing

Screen / feature	Reads (tables, views)	Actions (RPC, API, storage)
Home	`products`, `vendors`, `vendor_packages`, `categories`, `sectors`, `homepage_carousel_slides`	`get_vendor_spotlight` (vendor of the week card). Browse index is a light `products` query cached on device.
Search	`vendors`, `vendor_packages`	`search_products_full_v2` (products and services), `search_product_ids` (ID-only, used by Levi and "load more").
Vendors directory and map	`vendors`, `products`, `sectors`	`get_vendor_follower_counts`, `get_vendor_follow_states`, `toggle_vendor_follow`, distance is computed in the browser from vendor coordinates.
Wishlist	`favorites`, `package_favorites`	`add_to_cart`; add/remove rows in the two tables.

Product, package and storefront

Screen / feature	Reads (tables, views)	Actions (RPC, API, storage)
Product detail	`products`, `product_variants`, `favorites`, `product_comments`, `product_comment_likes`, `product_rating_stats`, `reviews`, `product_engagement_settings`	`increment_product_view`, `get_product_reviews`, `get_reviewable_order_for_product`, `get_favorite_counts`, `get_product_comment_counts`, `create_sale_order` (Buy now), `create_rental_order`, `add_to_cart`.
Service "name your price"	`service_offers`	`get_service_price_hint`, `create_service_offer`, `respond_service_offer` (accept / counter), `report_service_offer_payment`, `confirm_service_offer_payment`, `dispute_service_offer_payment`.
Package detail	`vendor_packages`, `package_items`, `package_favorites`, `package_comments`	`create_package_order`, `get_package_comment_counts`, `get_package_favorite_counts`.
Storefront	`vendors`, `products`, `vendor_packages`, `reviews`	`get_vendor_stats`, `increment_vendor_profile_view` (feeds "Views today"), `get_vendor_follow_state`, `toggle_vendor_follow`, `set_vendor_follow_preferences`, `get_or_create_conversation` (Message button).
Comments and likes	`product_comments`, `package_comments` and their `*_likes`	Plain inserts/deletes under RLS; realtime channel per product/package.

Cart and buying

Screen / feature	Reads (tables, views)	Actions (RPC, API, storage)
Cart	`cart_items` (joined to products)	`create_cart_order` (one vendor or all vendors at once), `add_to_cart`. Cart count is cached 15 s and refreshed after any write.

Edit order before vendor accepts	—	`edit_pending_order`, `set_order_buyer_location`.
Request an item	`equipment_requests`, `categories`, `sectors`	Insert into `equipment_requests`; vendors answer through `request_offers`.

Orders (customer and vendor)

Screen / feature	Reads (tables, views)	Actions (RPC, API, storage)
Customer orders / history	`orders`, `order_items`, `vendors`	`get_orders_buyer_names` (vendor side), live updates via realtime on `orders`.
Vendor orders	`orders`, `order_items`, `products`	`get_orders_buyer_names`; status buttons below.
Order details	`orders`, `order_items`, `payments`, `service_offers`, `conversations`, `users`, `vendors`, `order_notes`	`get_order_buyer_contact`, `add_order_note`, `send_order_reminder`, `vendor_suggest_order_quantity` / `buyer_respond_quantity_proposal`.
Status changes	—	`transition_order_status` (confirm, hand over, return, cancel by the right party), `/api/order-payout-action` (vendor decline / cancel, buyer cancel, refund), `buyer_confirm_received` (24 h flow), `/api/order-payout-action` (cancel / refund for card-paid orders).
Payments	`payments`	`buyer_report_manual_payment`, `vendor_confirm_reported_payment`, `vendor_dispute_reported_payment`, `vendor_confirm_unreported_payment`, `vendor_dispute_unreported_payment`, `record_offline_payment`, `/api/initiate-payment` + `/api/verify-payment` (online card).
Disputes	`disputes`	`raise_dispute` (+ evidence upload to bucket `dispute-evidence`), `vendor_respond_to_dispute`.
Reviews	`reviews`	Insert a review (one per completed order); a pop-up appears after completion.

Messages and calls

Screen / feature	Reads (tables, views)	Actions (RPC, API, storage)
Inbox	`conversations`, `messages`, `message_pins`, `blocked_users`	`list_my_conversations`, `toggle_conversation_pin`, `toggle_conversation_unread`, `mark_conversation_opened`, `conversation_is_open`, `get_conversation_other_party`, `get_conversation_presence`, chat lists: `list_my_chat_lists`, `create_chat_list`, `delete_chat_list`, `toggle_conversation_in_list`.
Chat	`messages`	Insert message (RLS + trigger enforce the open-chat rule), `edit_message`, `delete_message`, `toggle_message_reaction`; attachments to bucket `chat-attachments` (signed URLs).
Voice calls	`voice_calls`	`can_call_in_conversation`, `get_call_party`, `start_voice_call`, `update_voice_call`, `ring_callee_again`, `/api/turn` (ICE servers), broadcast channel `voicecall-<id>` for WebRTC signalling.
Suggestion box	`suggestions`	`submit_suggestion`, `get_platform_suggestions`, `get_vendor_suggestions`, `mark_suggestion_read`.

Vendor tools

Screen / feature	Reads (tables, views)	Actions (RPC, API, storage)

Become a vendor	`sectors`, `vendors`	`submit_vendor_application`, `get_or_create_custom_sector`, `create_vendor_sector` / `create_vendor_category`, documents to bucket `vendor-docs`, bank list via `/api/vendor-payments`.
My Shop (seller hub)	`vendors`, `products`, `orders`, `vendor_packages`	`get_vendor_stats`, `get_favorite_counts`, `get_vendor_follower_counts`, `vendor_set_storefront_active`, `get_vendor_commission_owed` / `_deadline` / `_breakdown`.
My Listings	`products`, `product_variants`, `monthly_listing_submissions`, `product_engagement_settings`	Insert/update/delete products, `save_product_variants`, `get_owned_product_variants`, `get_vendor_product_regions`, images to R2 via `/api/vendor-payments` (`r2-presign-upload`).
Billing (monthly or commission)	`commission_charges`	`set_vendor_billing_model`, `compute_monthly_billing_amount`, `submit_monthly_listing_payment` (+ receipt to `listing-payment-receipts`), `/api/vendor-payments` for card payment, drafts, subaccounts and bank details.
Earnings	`orders`	Reads only; charts are built in the browser.
Followers	`vendor_follows`	`get_vendor_followers` / `_v2`, `get_vendor_follower_network`.

Admin

Screen / feature	Reads (tables, views)	Actions (RPC, API, storage)
Vendors panel	`vendors`	`admin_set_vendor_status`, `admin_reject_vendor`, `admin_set_vendor_storefront_active`, `admin_force_unlock_vendor`, `admin_list_locked_vendors`, `admin_delete_vendor`, `admin_restore_vendor`, `admin_permanently_delete_vendor`.
Users and people	`users`	`admin_set_user_role`, `admin_suspend_user`, `admin_unsuspend_user`, `admin_get_pending_appeals`, `admin_deny_appeal`.
Disputes and payments	`disputes`, `orders`	`resolve_dispute`, `admin_confirm_reported_payment`, `admin_reject_reported_payment`, `admin_cancel_confirmed_overdue_order`, `admin_find_conversation`, `/api/order-payout-action`.
Finance	—	`admin_list_transactions`, `admin_list_vendor_payouts`.
Conversations	—	`admin_list_conversations`, `admin_get_conversation_participants`.
Monthly listings	`monthly_listing_submissions`	`admin_approve_monthly_listing`, `admin_reject_monthly_listing`, `admin_deactivate_monthly_listing`.
Points	`points_rules`	`admin_search_points_users`, `admin_adjust_points`, `admin_award_social_points`, `admin_recent_points_adjustments`, `admin_recent_points_skips`.

Content and support	`homepage_carousel_slides`, `support_requests`, `admin_actions`, `auth_activity_logs`	Banner images to R2 (fallback bucket `homepage-carousel`); support tickets with attachments in `support-attachments`; `admin_get_manual_password_reset_requests`, `admin_approve_/reject_manual_password_reset`.

Leaderboard, referrals and Levi

Screen / feature	Reads (tables, views)	Actions (RPC, API, storage)
Leaderboard	`points_rules`	`get_points_leaderboard`, `get_my_points_summary`, `get_my_points_activity`, `get_past_week_winners`, `get_vendor_spotlight`, `apply_referral_code`, `validate_referral_code`; realtime on `points_weekly`.
Levi chat	`ai_conversations`, `ai_messages`	`POST /api/chat`, `toggle_ai_message_reaction`, `search_ai_conversations`, `set_ai_conversation_shared`, `get_shared_ai_conversation`, `search_product_ids`.

Everywhere

These run on many screens and are the source of most “extra” calls: `enforce_commission_deadlines` and `get_vendor_commission_deadline` / `_owed` (the commission gate, vendors only), the notifications unread count (`notifications`), `touch_presence` / `clear_presence` / `touch_last_seen`, `record_auth_activity`.

11. Orders and payments in the UI

The database owns the rules (Backend §6.1–6.3). The frontend only decides which button to show and calls the matching function. This section is the quick link between the two.

Status to button

Order status	Customer sees	Vendor sees	Function called
`pending`	Edit, Cancel, Remind vendor	Accept (Confirm), Decline, suggest quantity	`transition_order_status`, `/api/order-payout-action` (decline), `edit_pending_order`, `send_order_reminder`
`confirmed`	Report payment (“I have paid”), Cancel	Confirm / dispute payment, Hand over	`buyer_report_manual_payment`, `vendor_confirm_reported_payment`, `vendor_dispute_reported_payment`, `transition_order_status`
`handed_over`	**I received this**, Report a problem	Waiting; confirm payment if unreported	`buyer_confirm_received`, `raise_dispute`, `vendor_confirm_unreported_payment`
`completed`	Review prompt, receipt	Earnings updated	`reviews` insert
`disputed`	Evidence and chat	Respond	`vendor_respond_to_dispute`; admin: `resolve_dispute`
`payment_unresolved`	Status note	Resolve or escalate	Escalated to admin by cron
`cancelled`	Reason shown	Reason shown	—

The tracker

The progress bar on Order details is built in the browser (`odStagesFor`, `odStageIndex`, `odIsPayAfter` in the `od-` code). Standard orders show four steps: Order placed → Confirmed → Handed over (Out for delivery) → Completed (Delivered). **Pay-after** orders (`payment_timing = "pay_after"`) show six steps: Placed → Confirmed → N% paid (the deposit percent, for example 30%; `payments` row `delivery_deposit_20`) → Out for delivery → Pay remaining (100 − N)% → Delivered. Which step is current comes from `order.status` plus the `order.payments` rows; no extra backend field is needed.

24-hour completion

After the vendor confirms the *final* payment, the database sets `orders.awaiting_confirmation_since` instead of completing the order. The customer sees a green **I received this** button (`confirmReceivedOrder` → `buyer_confirm_received`) and a note about the 24-hour limit. If they do nothing, `auto_complete_eligible_orders` (cron every 10 min) completes it. Rentals and negotiated service orders complete as before.

Cart checkout

`checkoutCartVendor` and `checkoutCartAll` call `create_cart_order`. The “all vendors” tick opens a confirmation sheet and then lists the resulting orders on the Orders page, one order per vendor.

12. Realtime, push and presence

Realtime channels

Channel name	Listens to	Used for
`notifications-<user>`	`notifications` (INSERT for this user)	Bell badge and list update at once.
`marketplace-<user>`	`messages` and related tables	Chat unread counts and inbox.
`orders-live-…`	`orders` (INSERT/UPDATE)	Order lists and status refresh.
`order-extras-…`	`payments`, notes	Payment status on order pages.
`requests-live-…`	Customer requests	Vendor requests inbox.
`conv-<conversationId>`	`messages` for the open chat	New, edited and deleted messages.
`product-comments-<id>`, `package-comments-<id>`	Comment tables	Live comments.
`voicecall-<callId>`	Broadcast (not a table)	WebRTC offer/answer/ICE signalling.
`lb-live-…`	`points_weekly`	Leaderboard moves live.

Realtime respects RLS: a user only receives rows they could also select. When changing a table that is listened to, make sure the realtime publication still includes it (Backend §5/§9).

Push notifications

1. The browser subscribes (`PUSH NOTIFICATIONS` section) using the public VAPID key and stores the subscription in `push_subscriptions`.
2. A database event creates a row in `notifications`. A trigger posts it to `/api/notify`, which sends email and web push (Backend §6.6).
3. `sw.js` receives the push. If a visible app window exists and it is not a call, it shows **nothing** (the app already shows the item). Calls and reminders have their own handling; tapping opens `target_view`.
4. While the app is open the heartbeat tells the server to skip system pushes (45 s window). Incoming calls always push, and the browser re-rings every few seconds (`ring_callee_again`, up to nine rings).

Settings has a “Call & message alerts check” with a test push (`levroTestPush` → `/api/notify`) and Android battery tips, because some Android phones delay pushes.

Voice calls

WebRTC audio. `start_voice_call` creates the call row and pushes the callee; offer/answer/ICE move over the `voicecall-<id>` broadcast channel; `/api/turn` returns TURN servers for hard networks; `update_voice_call` sets accepted/declined/ended; the chat shows call rows (missed, outgoing, incoming with duration).

13. Files and images

There are two storage systems, chosen by whether the file is public.

Kind	Where	How the browser uploads	Examples
Public images	Cloudflare R2	`uploadToR2(blob, key, type)` : asks `/api/vendor-payments` (`action: "r2-presign-upload"` , with the access token) for a presigned URL, then `PUT` s the file directly to R2 with a long `Cache-Control` . The public URL is stored in the database row.	Product photos, avatars, logos, covers, carousel banners.
Private files	Supabase Storage (signed URLs)	`sb.storage.from(bucket).upload(...)` , shown with `createSignedUrl` .	`chat-attachments` , `vendor-docs` , `listing-payment-receipts` , `dispute-evidence` , `support-attachments`
Older public buckets	Supabase Storage	Still read for old rows; new uploads use R2.	`avatars` , `vendor-logos` , `homepage-carousel` (fallback), `product-photos` (removal only)

Before upload, images are resized and compressed in the browser (`lib/compress-image.js`), which keeps storage and bandwidth low. Chat images also open in a small editor before sending. Admin has two one-time tools (R2 migration and size pass) that moved old images from Supabase Storage to R2; they can be removed once nobody needs them.

> **Migration note:** URLs of public images are stored in rows as full R2 URLs, so they survive a database move. Private files use signed URLs generated by Supabase; those must be re-created if Storage moves.

14. Levi, the AI assistant

Levi is the floating chat button (draggable, size and visibility are settings). The browser never talks to the AI provider directly.

Part	How it works
Chat	`fetchLeviReply` sends the conversation and the user's access token to `POST /api/chat` (Gemini). The function can use tools that read the signed-in user's own data (orders, listings, stats) and search the catalogue. The reply comes back as text.
History	`ai_conversations` and `ai_messages` (own rows only). Conversations can be shared by link (`set_ai_conversation_shared`; `sharedAiChat` view). The last conversation id is kept in `medris-ai-last-conversation`.
Products in chat	Levi resolves product mentions with `search_product_ids` and shows cards; "add to cart" actions call `add_to_cart`.
Voice	Android: records short 16 kHz clips cut at pauses (`LmChunkRecognizer`) and sends them to `/api/chat` with `mode: "transcribe"` (Gemini does the transcription). Other browsers use the Web Speech API. Text is cleaned and merged by `lmCleanVoice`, `lmJoinVoice`, `lmMergeResults`. Spoken replies use text-to-speech counted in `tts_usage`.
Limits	`increment_chat_usage` limits messages per day (`chat_usage`).

Levi's behaviour (instructions, tools, limits) lives in `api/chat.js`; the screens live in the `FLOATING AI CHAT` and `LEVI VOICE` sections.

15. Tables the browser touches directly

The browser reads or writes these 47 objects with `sb.from()`. All other tables (for example `orders` writes, `payments` writes, `points_*`, commission tables, `app_presence`) are only reachable through RPCs or the server. Operations shown are those that appear in the code; RLS decides which rows each role may actually touch (Backend §5, policy listing §8b).

Table / view	Operations used by the frontend	Table / view	Operations used by the frontend
`admin_actions`	insert/select	`package_items`	delete/insert/select
`ai_conversations`	delete/insert/select	`payments`	select
`ai_messages`	insert/select	`points_rules`	select
`auth_activity_logs`	select	`product_comment_likes`	delete/insert/select
`blocked_users`	delete/insert/select	`product_comments`	delete/insert/select
`cart_items`	delete/insert/select/update	`product_engagement_settings`	select/upsert
`categories`	select	`product_rating_stats (view)`	select
`commission_charges`	select	`product_variants`	delete/select
`conversations`	insert/select	`products`	delete/insert/select/update
`disputes`	select	`push_subscriptions`	upsert
`equipment_requests`	insert/select/update	`request_offers`	insert
`favorites`	delete/insert/select	`reviews`	insert/select
`homepage_carousel_slides`	delete/insert/select/update	`sectors`	select
`message_pins`	delete/insert/select	`service_offers`	select
`messages`	insert/select/update	`suggestions`	select
`monthly_listing_submissions`	select	`support_requests`	delete/insert/select/update
`notifications`	select/update	`users`	select/update/upsert
`order_items`	select	`vendor_follows`	select
`order_notes`	select	`vendor_packages`	delete/insert/select/update
`orders`	select	`vendor_profile_view_events`	select
`package_comment_likes`	delete/insert/select	`vendors`	delete/insert/select/update
`package_comments`	delete/insert/select	`voice_calls`	select
`package_engagement_settings`	select/upsert		

Key point for the backend engineer: **writes to `orders`, `payments`, `order_items`, `commission_*` and `points_*` never come directly from the browser**, so you can change those tables freely as long as the RPCs keep their names, arguments and return shapes.

16. Every backend function the frontend calls

147 distinct RPC names appear in `index.html`. If you change one (name, arguments, return columns), you must change the call sites too: search the name in `index.html`. Functions not on this list are used only by triggers, cron or the Vercel API.

Admin (36)

`admin_adjust_points` `admin_approve_manual_password_reset` `admin_approve_monthly_listing` `admin_award_social_points` `admin_cancel_confirmed_overdue_order` `admin_confirm_reported_payment` `admin_deactivate_monthly_listing` `admin_delete_inventory_product` `admin_delete_vendor` `admin_deny_appeal` `admin_find_conversation` `admin_force_unlock_vendor` `admin_get_conversation_participants` `admin_get_inventory` `admin_get_manual_password_reset_requests` `admin_get_pending_appeals` `admin_list_conversations` `admin_list_locked_vendors` `admin_list_transactions` `admin_list_vendor_payouts` `admin_permanently_delete_vendor` `admin_recent_points_adjustments` `admin_recent_points_skips` `admin_reject_manual_password_reset` `admin_reject_monthly_listing` `admin_reject_reported_payment` `admin_reject_vendor` `admin_restore_vendor` `admin_search_points_users` `admin_set_user_role` `admin_set_vendor_status` `admin_set_vendor_storefront_active` `admin_suspend_user` `admin_unsuspend_user` `resolve_dispute` `update_voice_call`

Orders & checkout (17)

`add_order_note` `add_to_cart` `buyer_confirm_received` `buyer_report_manual_payment` `buyer_respond_quantity_proposal` `create_cart_order` `create_package_order` `create_rental_order` `create_sale_order` `edit_pending_order` `get_order_buyer_contact` `get_orders_buyer_names` `get_reviewable_order_for_product` `send_order_reminder` `set_order_buyer_location` `transition_order_status` `vendor_suggest_order_quantity`

Payments, disputes & commission (18)

`compute_monthly_billing_amount` `confirm_service_offer_payment` `dispute_service_offer_payment` `enforce_commission_deadlines` `get_vendor_commission_breakdown` `get_vendor_commission_deadline` `get_vendor_commission_owed` `process_vendor_commission_hour_reminders` `raise_dispute` `record_offline_payment` `report_service_offer_payment` `set_vendor_billing_model` `submit_monthly_listing_payment` `vendor_confirm_reported_payment` `vendor_confirm_unreported_payment` `vendor_dispute_reported_payment` `vendor_dispute_unreported_payment` `vendor_respond_to_dispute`

Service offers (3)

`create_service_offer` `get_service_price_hint` `respond_service_offer`

Levi & usage (8)

`get_platform_suggestions` `get_shared_ai_conversation` `get_vendor_suggestions` `mark_suggestion_read` `search_ai_conversations` `set_ai_conversation_shared` `submit_suggestion` `toggle_ai_message_reaction`

Messages, calls & chat lists (22)

`can_call_in_conversation` `clear_presence` `conversation_is_open` `create_chat_list` `delete_chat_list` `delete_message` `edit_message` `get_call_party` `get_conversation_other_party` `get_conversation_presence` `get_or_create_conversation` `list_my_chat_lists` `list_my_conversations` `mark_conversation_opened` `ring_callee_again` `start_voice_call` `toggle_conversation_in_list` `toggle_conversation_pin` `toggle_conversation_unread` `toggle_message_reaction` `touch_last_seen` `touch_presence`

Follow, favourites & vendors (18)

`create_vendor_category` `create_vendor_sector` `get_favorite_counts` `get_package_favorite_counts` `get_vendor_follow_state` `get_vendor_follow_states` `get_vendor_follower_counts` `get_vendor_follower_network` `get_vendor_followers` `get_vendor_followers_v2` `get_vendor_product_regions` `get_vendor_spotlight` `get_vendor_stats` `increment_vendor_profile_view` `set_vendor_follow_preferences` `submit_vendor_application` `toggle_vendor_follow` `vendor_set_storefront_active`

Points, referrals & leaderboard (6)

`apply_referral_code` `get_my_points_activity` `get_my_points_summary` `get_past_week_winners` `get_points_leaderboard` `validate_referral_code`

Search, products & reviews (9)

`get_or_create_custom_sector` `get_owned_product_variants` `get_product_comment_counts` `get_product_reviews` `increment_product_view` `save_product_variants` `search_product_ids` `search_products_full` `search_products_full_v2`

Account & legal (9)

`accept_legal_terms` `complete_manual_password_reset` `create_notification` `get_legal_terms_consent` `get_manual_password_reset_status` `get_my_account_appeal` `record_auth_activity` `submit_account_appeal` `sync_current_account_role`

Other (1)

`get_package_comment_counts`

Vercel API endpoints the browser calls

Endpoint	Used for
`/api/auth-otp`	Admin login code, password change/set.
`/api/delete-account`	Sign-up (`/api/register`), email/phone check, account deletion, link-preview pages, `/api/turn` .
`/api/vendor-payments`	R2 upload signing, bank list, listing payment (card), commission payment, drafts, subaccount, bank details, one-time R2 tools.
`/api/initiate-payment` , `/api/verify-payment`	Online card payment start / check (Paystack).
`/api/order-payout-action`	Cancel / refund / release for card-paid orders.
`/api/notify`	Test push, call ring-again, decline-call from the notification.
`/api/chat`	Levi chat, suggestions, voice transcription.
`/api/turn`	TURN/ICE servers for calls.

17. Quick recipes

I want to…	Do this
Change a screen	Find its `renderX` in section 6, search for it, edit the **last** definition. Add CSS with a new prefix at the end of the last `<style>` block, then check light and dark mode.
Add a button that does a business action	Write a SECURITY DEFINER function in the database that checks `auth.uid()` and the rule, grant it to `authenticated`, then call `sb.rpc("name", {...})` from a new global function. Show `error.message` on failure.
Add a notification type	Create a database trigger that calls `create_notification(user, type, title, body, target_view, target_params, actor)`. Add the icon rule in `api/notify.js` (`pickNotificationIcon`). The bell and push need no frontend change; tapping navigates to `target_view`.
Add a new view	Add `case "myView": html = await renderMyView(); break;` in the `switch` inside `navigate`, write `renderMyView()`, and add it to `protectedRouteViews` if it needs sign-in.
Add a column to a card	Edit the card builder (`vendorCardHtml`, `vendorDirectoryCardHtml`, `productCardHtml` …); make sure the query selects the column; style in the matching prefix.
Change terms or privacy	`LEGAL PAGES` section; bump the version string in `accept_legal_terms` / `get_legal_terms_consent` to force re-acceptance.
Force all phones to update	Change `CACHE` in `sw.js` and deploy.
Add a table the browser reads	Enable RLS, add a SELECT policy, then use `sb.from()`. If it is a slow-changing table, add it to the cache `TTL` map in the safe-fetch script.
Reduce Supabase bandwidth	Select only needed columns, batch lookups into one RPC, cache with the TTL map, avoid polling (use realtime).

18. Frontend migration playbook (leaving Supabase)

This pairs with the migration playbook in the Backend manual (§12). The frontend is the harder half to change because it speaks Supabase's own protocols everywhere.

What binds the frontend to Supabase

Coupling	Where	Effort to replace
`supabase-js` client (`sb`)	One `createClient` call; 474 `.from()`, 224 `.rpc()` sites	Low if the new backend speaks the same API (PostgREST). High if rewriting each call.
Auth (GoTrue)	31 `sb.auth.*` sites; phone-alias emails; token used by `/api/*`	Medium. The access-token JWT is also verified by the Vercel functions.
Realtime	11 channels, 17 listeners, call signalling over broadcast	Medium. Needs a websocket service with row-change events and broadcast.
Storage (private files)	27 `.storage` sites, signed URLs	Low–medium. Move buckets to R2/S3 with signed URLs.
Safe-fetch cache regex	Matches `*.supabase.co/rest/v1/<table>`	Trivial: change the pattern.
Hard-coded values	`SUPABASE_URL`, anon key, `VAPID_PUBLIC_KEY`	Trivial.
Postgres RPC contract	147 function names, arguments and result shapes	None, if you keep Postgres and the same functions.

Recommended path

1. **Keep Postgres and keep the function names.** Any Postgres host works (Neon, RDS, a VPS). Restore the schema and data (Backend §12). Because business logic is in SQL functions, the frontend's 224 RPC calls keep working untouched.
2. **Put a PostgREST-compatible API in front** (PostgREST itself, or self-hosted Supabase), and a GoTrue-compatible auth (GoTrue, or Supabase Auth self-hosted). Then change only `SUPABASE_URL` and the anon key in `index.html`, plus the cache regex. This is the cheapest route: one constant instead of 700 call sites.
3. **Replace realtime.** Either self-hosted Supabase Realtime, or a small service that listens to Postgres `LISTEN/NOTIFY` and forwards events. Replace the 11 `sb.channel()` uses with the same event names so the handlers stay unchanged.
4. **Move private files** to R2/S3. Implement signed upload/download in an API function and swap the 27 storage calls for it.
5. **Re-point the Vercel functions** (`SUPABASE_URL`, service key, JWT verification) and the database-to-API webhook (Backend §6.6).
6. **Cut over** during a quiet hour: freeze writes, final dump and restore, switch the constants, deploy, bump `sw.js` so every phone reloads the new build.

If you choose a full rewrite of the data layer

Create one module (`api.js`) exposing the same methods the app uses today (`from().select()`, `rpc()`, `auth`, `storage`, `channel`) and implement them against your new backend. Replace the global `sb` with it. This keeps all call sites unchanged and lets you test against both backends.

Cut-over test checklist

- Sign up with email, sign up with phone, sign in with each, password reset (email and manual), admin OTP.
- Browse, search (with a misspelling), open a product, add to cart, checkout one vendor and all vendors.
- Full order: place → confirm → report payment → vendor confirms → hand over → "I received this"; also a pay-after order and the 24 h auto-complete.
- Chat (send, edit, delete, image), a voice call between two phones, push when the app is closed.
- Vendor: apply, approve (admin), add listing with photos, monthly/commission billing, earnings.
- Admin: every panel loads, a dispute is resolved, a points award works.
- Offline: open the app with no network; installed app on Android and iPhone.

19. Known issues and technical debt

Honest list, most important first. None of these stops the product working today.

Issue	Why it matters	Suggested fix
One 4.3 MB file with 80 scripts and 207 style blocks	Slow to edit, review and load; every change risks collisions; merge conflicts when two people edit.	Split into modules (router, pages, services, CSS) with a bundler. Do it page by page with screenshots before/after.
Layered CSS with `!important` and dark-mode overrides	New rules are often overridden; dark mode needs manual checks.	Remove superseded blocks per page; introduce design tokens; keep one rule per element.
Duplicate definitions of the same function	Editing the wrong copy has no effect (section 4).	Delete dead copies once the live one is confirmed.
HTML built as strings with `onclick=""` and global functions	Easy to leak globals; XSS is only prevented by remembering `esc()`.	Move to a small component approach; lint for un-escaped template values.
No automated tests	Regressions are found by hand.	Add Playwright smoke tests for sign-in, browse, checkout, chat, vendor listing.
Content of `.ts` files in `lib/` unused	Confusion about which compression file is live.	Keep `.js`, delete or wire up the `.ts` copies.
One-time admin tools (R2 migration / size pass), legacy inventory screen	Dead weight in the admin area.	Remove once confirmed unused.
Hidden admin tab rail kept only for compatibility	Admin navigation now uses a picker; the old rail stays in the DOM.	Remove after checking nothing reads it.
Native call alerts when the app is closed	Browser push is not enough for reliable ringing on every phone.	See `docs/NATIVE_CALLS_PLAN.md` (native shell, not built).
Large dependence on Supabase APIs	Migration cost (section 18).	Add the adapter module (`api.js`) first; it pays off even without migrating.

Gotchas learnt the hard way

- A CSS fix that “does nothing” is almost always specificity. Raise it at the end of the file and recheck dark mode.
- Edits to the wrong copy of a function do nothing (see the traps in section 4).
- One syntax error in any script block breaks the app; always run the syntax check.
- Installed apps keep old code until the service worker version changes.
- Phone login uses a hidden email alias; never show it to users and never change its format.
- A white flash or layout jump is usually the instant-shell render being replaced; check the `#app` swap.
- Android delays pushes with battery saver; test on a real phone, not only a desktop browser.

Appendix A: JavaScript section index

Each section starts with `// ---------- NAME ----------`. Line numbers are as of 7 October 2026 and drift as the file changes; search the name, not the number.

Line	Section	Line	Section
10250	SUPABASE	18754	IN-APP MESSAGING
10270	ACCOUNT INTEGRITY	18820	SUGGESTION BOX
10380	"INSTALL AS APP" PROMPT	19164	CHAT ROW SELECTION
10431	PUSH NOTIFICATIONS (on top of email, not instead of)	19230	CUSTOM CHAT LISTS (WhatsApp-style user-made lists)
10513	Call & message alerts check (Settings)	19377	CHAT SNAPSHOTS (instant open, works without data)
10585	FIRST-VISIT APP TOUR (live - walks the real pages, spotlights real elements)	19750	WHATSAPP-STYLE CHAT DATE SEPARATORS (buyer-vendor chat + Levi)
10989	UTILS	19808	CALL LOG ROWS IN THE CHAT (missed / outgoing / incoming + duration)
10990	AUTH IDENTIFIER HELPERS	19952	WHATSAPP-STYLE MESSAGE ACTIONS / SELECTION / REPLIES
11323	MAP (Leaflet + OpenStreetMap, loaded on demand)	21063	PACKAGE DETAIL
11686	AUDIO COMPATIBILITY	21235	PRODUCT DETAIL
11745	ACCESSIBILITY FONT SIZE	21942	VENDORS DIRECTORY
11761	LEVI FAB SIZE	21944	VENDORS PAGE: card + empty state
11771	LEVI VISIBILITY (hide the floating chat button entirely)	22453	STOREFRONT (public vendor page)
11797	TOASTS + CONFIRM DIALOG (replace native alert/confirm)	23101	REQUEST FORM HANDLERS
12104	THEME	23299	REQUEST EQUIPMENT (buyers post needs, vendors compete)
12139	ACCENT COLOR	23795	PRODUCT + PACKAGE SOCIAL INTERACTIONS
12241	SLIDE-IN DRAWERS	23857	VENDOR APPLICATION
12514	LONG-PRESS BATCH SELECTION	24129	VENDOR DASHBOARD
12575	NOTIFICATIONS (real, persisted, realtime)	24442	VENDOR WORKING HOURS (structured, replaces free-text opening_hours)
13140	LIVE ORDERS	24545	VENDOR: EDIT PROFILE
13324	AUTH	25256	VENDOR PRODUCTS
14009	DEDICATED LEVROMART SETTINGS PAGE	25260	MY LISTINGS (vendor catalogue page)
14041	MY DETAILS (any logged-in buyer/vendor's own contact info)	27031	VENDOR PACKAGES (bundle deals, e.g. an "Outreach Kit")
14131	BOTTOM NAV (mobile)	27323	VENDOR / BUYER ORDERS
14594	SUSPENDED ACCOUNT SCREEN	27346	ORDER STATS (today/week/month/all-time counts by status)
14715	SETTINGS EXTRAS	27853	ORDER HISTORY PAGE
14763	DEDICATED NOTIFICATIONS PAGE	27929	ORDER NOTES
14768	NAVIGATION	28057	ORDER DETAIL
15417	PERSISTENT ROUTE + SCROLL STATE	28082	CUSTOMER ORDER DETAILS (redesigned page)
15496	SWIPE PRODUCT / PACKAGE GALLERIES	28236	VENDOR ORDER DETAILS (redesigned page)
15519	HORIZONTAL BROWSE PREFERENCE	29039	QUANTITY SUGGESTIONS
15541	AUTO-SLIDING CAROUSELS (category row + horizontal product/vendor rails)	29605	PRODUCT RATINGS
15588	OFFLINE MARKETPLACE CACHE	29681	ADMIN
15688	HOME SEARCH SCROLL MOTION	29736	ADMIN: one-time R2 migration
15794	HOMEPAGE PROMOTIONAL CAROUSEL	29865	ADMIN: one-time R2 image size pass
16092	ADMIN LINK LIBRARY	30653	ADMIN: MANAGE ADMINS
16536	HOME (marketplace browse)	31498	INIT
17205	DEDICATED SEARCH PAGE	31511	LEGAL PAGES

17840	BROWSE EVERYTHING DATA (light index + 24-at-a-time rows)	31873	FLOATING AI CHAT (draggable)
18290	WISHLIST	32823	LEVI VOICE
18400	FOLLOWER NETWORK (vendors only)	34504	MODAL BACK-BUTTON / OUTSIDE-CLICK DISMISSAL
18440	CART (buy-and-sale items only, grouped by vendor; add/remove actions stay on the current page)		